

Principles of Functional Programming & Compiler Construction

From Lambda Calculus and Monadic Semantics to Stack Virtual Machines and Native Code

October 02, 2026

Foreword

This monograph provides a rigorous, self-contained mathematical and computational treatment of modern functional programming and compiler design. Grounded in Alonzo Church’s λ -calculus, Robin Milner’s type theory, and modern compiler architecture, these notes synthesizes some of the content presented during my computer science master’s option in 2025-2026.

Starting with the algebraic foundations of pure computation, pattern matching, laziness, and monadic category theory, the text transitions directly into practical compiler engineering: lexical scanning with Alex, LALR(1) parsing with Happy, intermediate representation design, stack machine architecture, symbolic assembly emission, and lowering to native x86-64 machine instructions.

Contents

Foreword	i
I Part I: Foundations of Functional Programming	1
1 Paradigms of Computation & Lambda Calculus	1
1.1 Idiomatic Expression: GCD Case Study	1
1.2 Syntax, Function Application, and Currying	2
1.3 Mathematical Purity and Referential Transparency	2
1.4 Strong Static Typing and Milner’s Dictum	3
2 Algebraic Data Types & Recursive Data Modeling	3
2.1 Sum Types and Constructors	3
2.2 Pattern Matching	3
2.3 Parametric Polymorphism	4
2.4 Typeclasses and Ad-Hoc Polymorphism	4
2.5 The Maybe Type and Handling Partiality	4
3 Evaluation Strategies & The Mechanics of Laziness	5
3.1 Strict vs. Non-Strict Evaluation	5
3.2 Thunks and Graph Reduction	5
3.3 Infinite Data Structures	5
3.4 Space Leaks and Strict Evaluation (<code>seq</code> , <code>\$!</code> , <code>BangPatterns</code>)	6
4 Category Theoretic Abstractions: Functors, Applicatives & Monads	6
4.1 Functors and Covariant Mapping	6
4.2 Applicative Functors	7

4.3	Monads and Sequencing Computations	7
4.4	The Maybe and List Monads	8
4.5	The <code>do</code> Notation Syntax Sugar	8
4.6	The <code>IO</code> Monad: Interacting with the Impure World	8
II	Part II: Front-End Architecture & Syntax Analysis	9
1	Compiler Architecture: From Source Strings to Target Executables	9
1.1	Classical Multi-Pass Pipeline	9
1.2	Formal Grammars and Chomsky Hierarchy	9
1.3	Shift-Reduce Parsing & LR Automata	9
2	Lexical & Syntactic Analysis: Alex & Happy in Practice	10
2.1	Structure of an Alex Lexer Specification (<code>.x</code>)	10
2.2	Structure of a Happy Parser Specification (<code>.y</code>)	11
2.3	Lexical and Syntactic Modernization: Parser Combinators (Parsec)	12
3	Abstract Syntax Trees & Expression Evaluation: The Calculator System	12
3.1	The Abstract Syntax Tree (AST) Data Type	12
3.2	Environments and Variable Lookup	13
3.3	Total Evaluation with the <code>Either</code> Error Monad	13
3.4	Interactive REPL (Read-Eval-Print Loop)	14
III	Part III: Back-End Construction & Code Generation	14
1	Stack Machine Architecture: The 17-Instruction Virtual Machine	14
1.1	Architectural Layout: Code, Data, and Stack Memories	15
1.2	The Symbolic Assembler (<code>asm</code>)	16
2	Code Generation: Compiling High-Level Languages to Stack Bytecode	17
2.1	Compiling Expressions: Infix to Postfix Stack Transformation	17
2.2	L-Values vs. R-Values in Variable Compilation	17
2.3	Control Flow Compilation: If-Then-Else and While Loops	18
2.3.1	Compiling <code>if (cond) then S1 else S2</code>	18
2.3.2	Compiling <code>while (cond) do S</code>	18
2.4	Array Indexing and Base-Offset Address Calculation	18
3	Functions, Activation Records & Runtime Memory Management	19

3.1	Function Calls and the Activation Record (Call Frame)	19
3.2	Parameter Passing Semantics	19
3.3	Compiling Recursive Functions on the Stack Machine	19
3.4	Tail Call Optimization (TCO)	20
4	Advanced Topics: Type Checking, Optimizations & Native Code Generation	20
4.1	Static Typing: The Hindley-Milner Type System & Algorithm W	20
4.2	Compiler Optimization Passes	21
4.3	Lowering to Native Target Code: The x86-64 NASM Backend	21
A	Course Laboratories, Milestones & Practical Exercises	23
A.1	Practical Problem Sets, Laboratory Exercises & Solutions	23
A.1.1	Problem Set 1: Lexing and Parsing Well-Parenthesized Expressions	23
A.1.1.1	Exercise 1.1: Maximum Parenthesis Depth	23
A.1.1.2	Exercise 1.2: Generalized Parse Tree Generation	23
A.1.2	Problem Set 2: The Extended Scientific Calculator (24 Milestones)	23
A.1.3	Problem Set 3: Pascal-to-Stack-Bytecode Compiler (42 Milestones)	24
A.1.3.1	Stage I: Scalar Expressions and I/O (Milestones 1–7)	24
A.1.3.2	Stage II: Variables and Sequential Statements (Milestones 8–11)	24
A.1.3.3	Stage III: Structured Control Flow (Milestones 12–15)	24
A.1.3.4	Stage IV: Functions and Procedures (Milestones 22–28)	25
A.1.3.5	Stage V: Advanced Optimizations (Milestones 29–42)	25
	Bibliography	26

Part I: Foundations of Functional Programming

Paradigms of Computation & Lambda Calculus

Computer science grounds computational processes into two foundational paradigms originating from the mid-20th century work of Alan Turing and Alonzo Church:

1. The Imperative Paradigm (Von Neumann Architecture):

- Programs are formulated as sequences of state transitions: $\sigma_0 \xrightarrow{c_1} \sigma_1 \xrightarrow{c_2} \dots \xrightarrow{c_n} \sigma_n$.
- The central mechanisms are mutable memory variables, assignment commands ($x := e$), iterative loops (**while**, **for**), and control transfers (**goto**, jumps).
- The program specifies **how** a machine modifies its registers and addressable random-access memory (RAM).

2. The Declarative / Functional Paradigm (λ -Calculus):

- Programs are mathematical expressions denoting values.
- Computation consists of term rewriting and substitution (specifically β -reduction in Alonzo Church's λ -calculus):

$$(\lambda x. e_1) e_2 \xrightarrow{\beta} e_1[x \mapsto e_2] \quad (0.1)$$

- Computation is stateless and free of uncontrolled side effects: the programmer specifies **what** relationships hold and what values are defined, rather than managing CPU cycles and memory cells.

Theorem 1 (Church-Turing Thesis). *Every effectively calculable function over the natural numbers can be computed by a Turing Machine, and equivalently by an expression in the untyped λ -calculus. Both models are Turing-complete and possess identical computational power.*

Idiomatic Expression: GCD Case Study

To illustrate the philosophical divide between how imperative code manages state and how functional code declares equations, consider the Euclidean algorithm for computing the Greatest Common Divisor (gcd) of two integers $a, b \in \mathbb{N}$.

Imperative Python:

```
def gcd(x: int, y: int) -> int:
    while y != 0:
        r = x % y
        x = y
        y = r
    return x
```

In Python, computation proceeds by mutating the bindings of local memory locations x , y , and r through successive clock steps of the loop until the condition $y = 0$ is realized.

Declarative Haskell:

```
gcd :: Integral a => a -> a -> a
gcd x 0 = x
gcd x y = gcd y (x `mod` y)
```

In Haskell, the algorithm is expressed as a set of equational identities matching the mathematical formulation:

$$\text{gcd}(x, y) = \begin{cases} x & \text{if } y = 0 \\ \text{gcd}(y, x \bmod y) & \text{if } y \neq 0 \end{cases} \quad (0.2)$$

There is no mutable variable, no hidden intermediate register, and no pointer state.

Syntax, Function Application, and Currying

In conventional programming languages (such as C, Java, or Python), invoking a function f with arguments x and y requires parenthesized comma-separated tuples: $\mathbf{f}(x, y)$.

In Haskell, OCaml, and the formal λ -calculus:

- Function application is represented by mere juxtaposition: $\mathbf{f} \ x \ y$.
- Parentheses serve exclusively for grouping expressions and defining precedence.
- Function application is strictly **left-associative**:

$$f \ x \ y \equiv ((f \ x) \ y) \quad (0.3)$$

Definition 1 (Currying). Let X, Y, Z be types. Currying is the natural bijection between functions operating on Cartesian product pairs and functions returning intermediate functions:

$$\text{curry} : (X \times Y \rightarrow Z) \cong (X \rightarrow (Y \rightarrow Z)) \quad (0.4)$$

In Haskell, every multi-argument function is curried by default. The type arrow $\rightarrow$ is **right-associative**:

$$A \rightarrow B \rightarrow C \equiv A \rightarrow (B \rightarrow C) \quad (0.5)$$

Example. (Stepwise Curried Evaluation). Consider the expression `gcd 2 3`:

1. `(gcd 2)` evaluates to an intermediate unary function of type `Int -> Int` which has captured the first parameter 2.
2. `((gcd 2) 3)` evaluates this intermediate function on the second argument 3, yielding the final integer result 1.

Mathematical Purity and Referential Transparency

Definition 2 (Pure Function). A function $f : A \rightarrow B$ is **pure** if and only if:

1. When invoked with identical arguments $x_1 = x_2 \in A$, it always produces an identical output $f(x_1) = f(x_2) \in B$.
2. It causes no observable side effect (such as mutating global variables, performing hidden I/O, modifying arguments in place, or reading clock/hardware states).

A direct consequence of purity is **referential transparency**: any expression can be replaced by its evaluated result without altering the observable behavior or semantic correctness of the enclosing program.

Remark. (Optimization Advantages of Referential Transparency). Because pure functions do not depend on external state or order of execution:

- The compiler can execute subexpressions in parallel without race conditions or locks.
- Pure expressions can be memoized, reordered, commoned up (Common Subexpression Elimination), or lifted out of loops.
- In production GHC compilers, compile-time transformations such as **Generalized Stream Fusion** eliminate intermediate data structures: Chains of transformations like

$$\text{map } f \circ \text{map } g \circ \text{filter } p \quad (0.6)$$

are fused into a single tight loop, avoiding intermediate allocations.

Strong Static Typing and Milner's Dictum

Haskell enforces a non-negotiable Hindley-Milner static type system. Types are resolved and checked entirely at compile time.

Theorem 2 (Robin Milner's Polymorphism Principle (1978)). *“Well-typed programs cannot go wrong.” In a type-sound language, a program that successfully satisfies the static type checker will never encounter undefined run-time machine crashes, invalid memory interpretations, or type confusion errors at execution time.*

Algebraic Data Types & Recursive Data Modeling

The core expressive power of functional programming stems from **Algebraic Data Types (ADTs)**. An ADT is defined by the `data` keyword and allows constructing sum types (disjoint unions) and product types (records/tuples).

Sum Types and Constructors

Definition 3 (Algebraic Data Type). *An Algebraic Data Type T is declared by specifying its set of data constructors $C_1, \dots, C_k$, each accepting zero or more field types:*

$$\text{data } T = C_1 \tau_{1,1} \dots \tau_{1,m_1} \mid \dots \mid C_k \tau_{k,1} \dots \tau_{k,m_k} \quad (0.7)$$

Values of type T are constructed inductively by applying one of the constructors C_i to appropriate argument values.

Consider the inductive definition of an integer linked list:

```
data IntList = Nil | Cons Int IntList
```

Here:

- `Nil` is the base constructor (nullary, representing the empty list $\emptyset$).
- `Cons` is the inductive constructor, taking an `Int` head and a recursive tail of type `IntList`.

Pattern Matching

Computation over ADTs is conducted via **pattern matching**, which deconstructs algebraic values into their constituent components based on constructor forms. Pattern matching can be structured via equational clauses or through `case ... of` blocks.

```
-- Equational pattern matching:
safeHead :: IntList -> Int
safeHead Nil      = error "safeHead: empty list has no head"
safeHead (Cons x _) = x

-- Equivalent case analysis:
safeHeadCase :: IntList -> Int
safeHeadCase l = case l of
  Nil      -> error "safeHeadCase: empty list"
  Cons x _ -> x
```

The `case l of` construct inspects the evaluated form (the **scrutinee**) of `l`, testing each pattern sequentially from top to bottom and evaluating the expression corresponding to the first matching constructor branch. The underscore wildcard `_` matches any subterm without binding an identifier, discarding unneeded values.

Parametric Polymorphism

Writing distinct list implementations for `Int`, `Double`, and `String` violates software reusability and leads to massive code duplication. In Hindley-Milner type systems, we introduce **type variables** (designated by lowercase identifiers like `a`, `b`) to obtain universally quantified generic data structures:

```
data List a = Nil | Cons a (List a)
```

Here, `List` is a **type constructor** of kind $* \rightarrow *$. It takes any ground type `a` and yields the concrete type `List a`.

Typeclasses and Ad-Hoc Polymorphism

While parametric polymorphism operates uniformly across all types without inspecting their structure, many operations require specific capabilities (e.g., equality testing, numeric addition, serialization). Haskell resolves this via **typeclasses**.

Definition 4 (Typeclass). A typeclass defines an abstract interface comprising method signatures:

```
class Eq a where
  (==) :: a -> a -> Bool
  (/=) :: a -> a -> Bool
  x /= y = not (x == y)
```

A concrete type `T` becomes an instance of a typeclass by providing implementations for its minimal complete definition:

```
instance Eq a => Eq (List a) where
  Nil      == Nil      = True
  Cons x xs == Cons y ys = (x == y) && (xs == ys)
  _        == _        = False
```

Important standard typeclasses in Haskell:

- `Eq`: Supports equality tests (`==`) and (`/=`).
- `Ord`: Extends `Eq` with total ordering relations (`<`, `<=`, `>`, `>=`, `compare`).
- `Show`: Provides string rendering (`show :: a -> String`).
- `Read`: Parses textual representations into typed values (`read :: Read a => String -> a`).
- `Num`, `Integral`, `Fractional`, `Floating`: Structure algebraic number hierarchies.

The Maybe Type and Handling Partiality

Functions in classical mathematics and programming are frequently **partial**: their domain of definition does not cover the entire input type.

- Example 1: Integer division by zero (`div x 0`).
- Example 2: Extracting the head of an empty list (`head []`).
- Example 3: Looking up a non-existent key in an associative map (`lookup k m`).

In imperative languages, partiality is typically handled by returning sentinel null pointers (e.g., `NULL`, `None`) or throwing unhandled run-time exceptions, which undermine referential transparency and cause crashes.

Haskell enforces explicit type-level partiality using the **Maybe** type:

```
data Maybe a = Nothing | Just a
```

A total head function is thus specified as:

```
totalHead :: List a -> Maybe a
totalHead Nil      = Nothing
totalHead (Cons x _) = Just x
```

The return type `Maybe a` informs both the compiler and downstream callers that failure is a possible outcome. The programmer is forced by the type checker to match and handle both `Nothing` and `Just` branches, completely eliminating null pointer exceptions at compile time.

Evaluation Strategies & The Mechanics of Laziness

Haskell is a **purely functional, non-strict** programming language evaluated by **call-by-need** (lazy evaluation with graph sharing). Understanding how laziness operates at the machine level is crucial for mastering complexity, termination, and space usage.

Strict vs. Non-Strict Evaluation

Let e be a compound expression containing a subterm e_1 .

- **Strict Evaluation (Call-by-Value):** Subterms $e_1, e_2, \dots$ are evaluated to values **before** the function f is invoked. If evaluating e_1 diverges ($e_1 = \perp$), the entire function call $f(e_1)$ diverges immediately.
- **Non-Strict Evaluation (Call-by-Name / Call-by-Need):** Function arguments are passed unevaluated into the function body. An argument is evaluated if and only if its value is strictly demanded by a primitive operation or pattern match.

Definition 5 (Bottom (\$bot\$) and Non-Strictness). Let $\perp$ (“bottom”) denote a non-terminating computation (an infinite loop) or an explicit runtime abort (error “boom”). A function f is **strict** if:

$$f(\perp) = \perp \tag{0.8}$$

A function f is **non-strict** if there exists an input such that $f(\perp) \neq \perp$.

Example. (Non-Strict Evaluation of Conditionals and Constants). Consider the constant function `const :: a -> b -> a` defined by `const x y = x`:

```
const 42 (error "divergence!") -- Evaluates strictly to 42!
```

Because the second argument is never scrutinized by the function, the error branch is never evaluated.

Thunks and Graph Reduction

Under non-strict semantics, an unevaluated expression is represented in heap memory as a **thunk** (a heap allocation containing a code pointer to the suspended computation and pointers to its free variables).

When an expression’s value is demanded:

1. The CPU forces the thunk by executing its suspension code.
2. **Call-by-Need Sharing:** The thunk updates its heap cell in place with the evaluated result (an indirect node or constructor). Subsequent accesses read the memoized value directly in $\mathcal{O}(1)$ time, avoiding duplicate evaluation.

Infinite Data Structures

Lazy evaluation decouples **data generation** from **data consumption**. A generator can define an infinite stream, while the consumer scrutinizes only the necessary prefix.

Consider the native list type `[a]`:

```
-- Standard infinite sequence of Fibonacci numbers:
fibs :: [Integer]
fibs = 0 : 1 : zipWith (+) fibs (tail fibs)
```

Evaluating `take 10 fibs` computes only the first 10 elements. The remainder of the infinite stream remains unallocated in memory.

Similarly, consider the infinite list of natural numbers `[1..]`:

```
-- take 5 [1..] evaluates step-by-step:
take 5 [1..] ==> [1, 2, 3, 4, 5]
```

Space Leaks and Strict Evaluation (`seq`, `$!`, `BangPatterns`)

While laziness enables modularity and infinite data structures, excessive lazy accumulation of unevaluated thunk chains can lead to critical memory exhaustion known as **space leaks**.

Consider the naive definition of list summation:

```
sumNaive :: Num a => [a] -> a
sumNaive = foldl (+) 0
```

When evaluating `foldl (+) 0 [1..1000000]`, Haskell does not compute numbers on the fly; instead, it builds a massive thunk tree:

$$(\dots((0 + 1) + 2) + \dots + 1000000) \quad (0.9)$$

Evaluating this deeply nested tree causes a stack overflow.

To enforce strict evaluation, Haskell provides the primitive `seq`:

```
seq :: a -> b -> b
```

`seq a b` forces the evaluation of `a` to **Weak Head Normal Form (WHNF)** before returning `b`.

The strictly accumulating fold `foldl'` in `Data.List` uses strictness:

```
foldl' :: (b -> a -> b) -> b -> [a] -> b
foldl' f !acc [] = acc
foldl' f !acc (x:xs) = foldl' f (f acc x) xs
```

Here, the bang pattern `!acc` forces the intermediate accumulator to be evaluated at each step, ensuring $\mathcal{O}(1)$ auxiliary space complexity.

Category Theoretic Abstractions: Functors, Applicatives & Monads

One of the central design challenges in pure functional programming is managing effects—such as partiality, non-determinism, state accumulation, and input/output—while retaining mathematical purity and equational reasoning. Haskell accomplishes this through abstractions rooted in Category Theory: **Functors**, **Applicative Functors**, and **Monads**.

Functors and Covariant Mapping

Definition 6 (Functor Typeclass). A type constructor F of kind $* \rightarrow *$ is a **Functor** if it implements the mapping operator `fmap`:

```
class Functor f where
  fmap :: (a -> b) -> f a -> f b
  (<$>) :: (a -> b) -> f a -> f b
  (<$>) = fmap
```

A valid functor must satisfy the two categorical **Functor Laws**:

1. **Identity Law**:

$$\text{fmap id} = \text{id} \quad (0.10)$$

2. **Composition Law**:

$$\text{fmap } (f \circ g) = (\text{fmap } f) \circ (\text{fmap } g) \quad (0.11)$$

Example. (Functor Instances).

1. **Maybe Functor:**

```
instance Functor Maybe where
  fmap _ Nothing = Nothing
  fmap f (Just x) = Just (f x)
```

2. **List Functor:**

```
instance Functor [] where
  fmap = map
```

Applicative Functors

While Functors allow lifting unary functions over a context f , they cannot directly apply a function encapsulated inside a context $f(a \rightarrow b)$ to an argument inside fa . This capability is provided by **Applicative Functors**.

Definition 7 (Applicative Functor). A functor F is an **Applicative Functor** if it provides a unit constructor `pure` and sequential application `<*>`:

```
class Functor f => Applicative f where
  pure  :: a -> f a
  (<*>) :: f (a -> b) -> f a -> f b
```

Monads and Sequencing Computations

When computations produce intermediate values that dictate subsequent contextual actions, simple applicative chaining is insufficient. We require dynamic sequential composition: this is the essence of a **Monad**.

Definition 8 (Monad Typeclass). A type constructor M of kind $* \rightarrow *$ is a **Monad** if it provides `return` and the sequencing bind operator `>>=`:

```
class Applicative m => Monad m where
  return :: a -> m a
  return = pure

  (>>=) :: m a -> (a -> m b) -> m b
  (>>)  :: m a -> m b -> m b
  m1 >> m2 = m1 >>= (\_ -> m2)
```

Every lawful monad must satisfy the three fundamental **Monad Laws**:

1. **Left Identity:**

$$(\text{return } x) \gg= k \equiv k x \quad (0.12)$$

2. **Right Identity:**

$$m \gg= \text{return} \equiv m \quad (0.13)$$

3. **Associativity:**

$$(m \gg= f) \gg= g \equiv m \gg= (\lambda x. f(x) \gg= g) \quad (0.14)$$

Theorem 3 (Typeclass Hierarchy Theorem). Every Monad is an Applicative Functor, and every Applicative Functor is a Functor:

$$\text{Monad } m \Rightarrow \text{Applicative } m \Rightarrow \text{Functor } m \quad (0.15)$$

The Maybe and List Monads

1. Maybe as a Monad (Exception Handling without Traps):

```
instance Monad Maybe where
  Nothing >=> _ = Nothing
  Just x  >=> f = f x
```

If any intermediate subcomputation fails and produces `Nothing`, the failure propagates directly to the end of the chain without invoking subsequent steps.

2. Lists as a Monad (Non-Deterministic Computation):

```
instance Monad [] where
  xs >=> f = concat (map f xs)
```

The list monad represents non-deterministic branching computations where each step may produce zero, one, or multiple candidate results.

The `do` Notation Syntax Sugar

Chaining raw `>=>` and lambda expressions can quickly degrade readability:

```
action1 >=> (\x1 ->
  action2 >=> (\x2 ->
    mk_action3 x1 x2))
```

Haskell introduces the `do` notation as purely syntactic sugar for monadic pipelines:

```
do
  x1 <- action1
  x2 <- action2
  mk_action3 x1 x2
```

The `IO` Monad: Interacting with the Impure World

Because Haskell is purely functional, functions cannot directly perform side effects like reading keystrokes or printing to the console without violating referential transparency.

The solution is the `IO` Monad:

```
putStrLn :: String -> IO ()
getLine  :: IO String
```

A value of type `IO a` is not an `a`; it is a **pure computation recipe** that, when executed by the Haskell runtime environment, performs I/O and yields a result of type `a`.

Example. (Interactive Echo Pipeline). Consider constructing an interactive terminal echo program:

```
-- Monadic bind composition:
echo :: IO ()
echo = getLine >=> putStrLn

-- Equivalent do block:
echoDo :: IO ()
echoDo = do
  line <- getLine
  putStrLn ("Echo: " ++ line)
```

Notice that you cannot simply write `putStrLn (getLine)`: the static type checker catches the type mismatch between `IO String` and `String`, guaranteeing that untrusted impure interactions cannot infiltrate pure functions without being tracked in the type system.

Part II: Front-End Architecture & Syntax Analysis

Compiler Architecture: From Source Strings to Target Executables

A compiler is a software pipeline that transforms source code written in a high-level language into an equivalent target representation (such as stack bytecode or native machine assembly).

Classical Multi-Pass Pipeline

The compilation process is structured into distinct, modular phases:

1. **Lexical Analysis (Scanning / Lexing)**: Converts a raw, unstructured stream of characters $c_1, c_2, \dots \in \Sigma^*$ into a sequence of discrete lexical tokens $T_1, T_2, \dots \in \mathcal{T}^*$. Comments and extraneous whitespace are filtered out, and line/column source positions are attached.
2. **Syntactic Analysis (Parsing)**: Consumes the token stream and verifies conformance with a formal context-free grammar (CFG). It generates a hierarchical tree structure: the **Abstract Syntax Tree (AST)**.
3. **Semantic Analysis & Type Checking**: Traverses the AST to verify scope rules, resolve identifier bindings in symbol tables, and enforce type constraints.
4. **Intermediate Code Generation & Optimization**: Translates the typed AST into an intermediate representation (IR), applying constant folding, dead-code elimination, and register/stack optimization.
5. **Code Generation (Target Emission)**: Emits target assembly instructions, assigns memory layouts, and resolves branch labels.

Formal Grammars and Chomsky Hierarchy

Definition 9 (Context-Free Grammar (CFG)). A Context-Free Grammar is a 4-tuple $G = (V, \Sigma, R, S)$ where:

- V is a finite set of **non-terminal** characters (syntactic categories).
- Σ is a finite set of **terminal** characters (tokens), disjoint from V .
- $R \subseteq V \times (V \cup \Sigma)^*$ is a finite set of **production rules**, written $A \rightarrow \alpha$.
- $S \in V$ is the distinguished **start symbol**.

Grammars are commonly rendered in **Backus-Naur Form (BNF)**:

$$\text{Expr} \rightarrow \text{Expr} \ + \ \text{Term} \mid \text{Term} \tag{0.16}$$

$$\text{Term} \rightarrow \text{Term} \ * \ \text{Factor} \mid \text{Factor} \tag{0.17}$$

$$\text{Factor} \rightarrow (\ \text{Expr} \) \mid \text{number} \tag{0.18}$$

Shift-Reduce Parsing & LR Automata

Deterministic bottom-up parsers, including those generated by **Yacc** and **Happy**, belong to the family of $LR(k)$ parsers (**Left-to-right scanning, Rightmost derivation in reverse**, with k tokens of lookahead, typically $k = 1$ or LALR(1)).

An LR parser maintains an internal stack of states and symbols and operates via four fundamental actions:

1. **Shift:** Read the current lookahead token from the input stream, push it along with the new DFA state onto the parse stack, and advance the input cursor.
2. **Reduce:** When the top of the stack matches the right-hand side of a production rule $A \rightarrow \alpha$, pop $|\alpha|$ symbols from the stack and push the non-terminal A , triggering the associated semantic action.
3. **Accept:** The input is exhausted, and the start symbol S occupies the stack; parsing successfully terminates.
4. **Error:** No valid shift or reduce transition exists for the current state and lookahead token.

Remark. (Grammar Ambiguities: Shift/Reduce and Reduce/Reduce Conflicts). When a grammar admits multiple parse trees for the same input string:

- **Shift/Reduce Conflict:** The parser cannot decide whether to shift the next token or reduce the current stack items (e.g., the classic “dangling-else” dilemma).
- **Reduce/Reduce Conflict:** Two distinct grammar rules simultaneously match the current stack top, indicating fundamental syntactic ambiguity or poor grammar factoring.

Resolving these conflicts requires declaring operator precedence (`%left`, `%right`, `%nonassoc`) or refactoring non-terminals.

Lexical & Syntactic Analysis: Alex & Happy in Practice

In the Haskell ecosystem, lexical analyzers and syntactic parsers are generated automatically from declarative domain-specific specifications using **Alex** (the Haskell analog of Lex/Flex) and **Happy** (the Haskell analog of Yacc/Bison).

Structure of an Alex Lexer Specification (.x)

An Alex specification file contains three primary sections:

1. **Top Haskell Block** (`{ ... }`): Module header declaring exports (typically `Token(..)` and `alexScanTokens`) and necessary imports.
2. **Alex Configuration & Rules:**
 - Wrappers: `%wrapper "basic"` (simple string-to-token scanner) or `%wrapper "posn"` (tracks byte/character line and column positions via `AlexPn`).
 - Macro definitions for character sets: `$digit = [0-9]`, `$alpha = [a-zA-Z]`.
 - Token rules declared under `tokens :-: <state> regexp { \s -> TokenValue }`
3. **Bottom Haskell Block** (`{ ... }`): Definition of the `Token` algebraic data type, helper conversion functions, and error formatters.

Example. (Alex Scanner with Source Position Tracking).

```
{
module CalcLexer (Token(..), TokenVal(..), scanTokens) where
}

%wrapper "posn"

$digit = 0-9
$alpha = [a-zA-Z]

tokens :-
  $white+           ; -- Discard whitespace
  ";"              { \p _ -> TK p TK_SEMI }
  "+"              { \p _ -> TK p TK_PLUS  }
  "-"              { \p _ -> TK p TK_MINUS }
  "*"              { \p _ -> TK p TK_MUL   }
  "/"              { \p _ -> TK p TK_DIV   }
```

```

"(" { \p _ -> TK p TK_LPAREN }
")" { \p _ -> TK p TK_RPAREN }
"=" { \p _ -> TK p TK_ASSIGN }
$digit+(\.$digit+)?([eE](\+|-)?$digit+)? { \p s -> TK p (TK_DOUBLE (read s)) }
$alpha[$alpha $digit]* { \p s -> TK p (TK_IDENT s) }

{
data TokenVal
  = TK_SEMI | TK_PLUS | TK_MINUS | TK_MUL | TK_DIV
  | TK_LPAREN | TK_RPAREN | TK_ASSIGN
  | TK_DOUBLE Double
  | TK_IDENT String
  deriving (Eq, Show)

data Token = TK AlexPosn TokenVal deriving (Eq, Show)

scanTokens :: String -> [Token]
scanTokens = alexScanTokens
}

```

Structure of a Happy Parser Specification (.y)

A Happy grammar specification consists of:

- Configuration Directives:**
 - `%name <function_name>`: Name of the generated parsing function (e.g., `parseCalc`).
 - `%tokentype { <Haskell_Token_Type> }`: Type of incoming tokens.
 - `%error { <error_handler> }`: Function called upon syntactic failure (`parseError :: [Token] -> a`).
- Token Declarations (`%token`):** Maps internal terminal names used in the grammar to constructor patterns over `Token`.
- Precedence & Associativity Directives:**
 - `%left '+' '-'`: Left-associative additive operators.
 - `%left '*' '/'`: Higher precedence multiplicative operators.
 - `%right '^`: Right-associative exponentiation.
- Grammar Rules (`%`):** Non-terminal production rules annotated with semantic action blocks in `{ ... }`. Within actions, `$1`, `$2`, `dots`, `$n` refer to the synthesized attributes of the n symbols on the right-hand side.

Example. (Happy Grammar with Operator Precedence and AST Construction).

```

{
module CalcParser (parseExpr) where
import CalcLexer
import ArithAST
}

%name parseExpr
%tokentype { Token }
%error { parseError }

%token
  double { TK _ (TK_DOUBLE $$) }
  ident  { TK _ (TK_IDENT $$) }
  '+'    { TK _ TK_PLUS }
  '-'    { TK _ TK_MINUS }
  '*'    { TK _ TK_MUL }

```

```

'/'      { TK _ TK_DIV }
'('      { TK _ TK_LPAREN }
')'      { TK _ TK_RPAREN }

%left '+' '-'
%left '*' '/'
%left NEG

%%

Expr : Expr '+' Expr      { BinOp Add $1 $3 }
    | Expr '-' Expr      { BinOp Sub $1 $3 }
    | Expr '*' Expr      { BinOp Mul $1 $3 }
    | Expr '/' Expr      { BinOp Div $1 $3 }
    | '-' Expr %prec NEG { UnOp Neg $2 }
    | '(' Expr ')'        { $2 }
    | double              { LitDouble $1 }
    | ident               { VarRef $1 }

{
parseError :: [Token] -> a
parseError [] = error "Parse error: Unexpected end of input"
parseError (TK (AlexPn _ line col) val : _) =
    error $ "Syntax error at line " ++ show line ++ ", column " ++ show col ++ " on token "
    ++ show val
}

```

Lexical and Syntactic Modernization: Parser Combinators (Parsec)

While Alex and Happy remain standard tools for deterministic LALR(1) grammars, modern compiler engineering frequently leverages monadic **parser combinators** such as Haskell's **Parsec** or **Megaparsec**.

- **Advantage:** Grammars are written directly in native Haskell code without an external preprocessor phase.
- **Typing:** Parsing actions are typed statically by GHC immediately, eliminating obscure post-code-generation type errors.
- **Error Diagnostics:** Combinator parsers retain full monadic context, enabling highly descriptive error reports with expected symbol sets.

Abstract Syntax Trees & Expression Evaluation: The Calculator System

Before generating target assembly or virtual machine instructions, a compiler represents input programs as **Abstract Syntax Trees (ASTs)**. We develop a complete, incremental scientific calculator language supporting floating-point arithmetic, symbol environments, unary/binary operations, and transcendental functions.

The Abstract Syntax Tree (AST) Data Type

We define an expressive recursive data type **AExpr** modeling arithmetic expressions:

```

module ArithExpression where

type Ident = String

data BinOp = Add | Sub | Mul | Div | Pow deriving (Eq, Show)
data UnOp  = Neg | Sin | Cos | Tan | Exp | Log | Sqrt deriving (Eq, Show)

```

```
data AExpr
  = LitDouble Double
  | VarRef Ident
  | Binary BinOp AExpr AExpr
  | Unary UnOp AExpr
  | Assign Ident AExpr
  | Sequence [AExpr]
  deriving (Eq, Show)
```

Environments and Variable Lookup

Evaluating expressions containing variables requires an **evaluation environment** mapping identifier strings to concrete floating-point numbers:

```
type Env = [(Ident, Double)]

lookupVar :: Ident -> Env -> Either String Double
lookupVar name [] = Left ("Undefined variable identifier: " ++ name)
lookupVar name ((k, v):xs)
  | name == k = Right v
  | otherwise = lookupVar name xs

extendEnv :: Ident -> Double -> Env -> Env
extendEnv name val env = (name, val) : filter (\(k, _) -> k /= name) env
```

Total Evaluation with the Either Error Monad

To handle run-time anomalies—such as division by zero, logarithms of non-positive numbers, or accessing unbound variables—we formulate the evaluator using `Either String Double`:

```
evalExpr :: AExpr -> Env -> Either String (Double, Env)
evalExpr (LitDouble d) env = Right (d, env)

evalExpr (VarRef x) env = do
  val <- lookupVar x env
  return (val, env)

evalExpr (Assign x expr) env = do
  (val, env') <- evalExpr expr env
  let newEnv = extendEnv x val env'
  return (val, newEnv)

evalExpr (Unary Neg e) env = do
  (v, env') <- evalExpr e env
  return (-v, env')

evalExpr (Unary Sqrt e) env = do
  (v, env') <- evalExpr e env
  if v < 0
  then Left "Domain error: square root of negative number"
  else return (sqrt v, env')

evalExpr (Binary Div e1 e2) env = do
  (v1, env1) <- evalExpr e1 env
  (v2, env2) <- evalExpr e2 env1
  if v2 == 0
  then Left "Arithmetic error: Division by zero"
  else return (v1 / v2, env2)
```

```
evalExpr (Binary Add e1 e2) env = do
  (v1, env1) <- evalExpr e1 env
  (v2, env2) <- evalExpr e2 env1
  return (v1 + v2, env2)
```

```
evalExpr (Binary Mul e1 e2) env = do
  (v1, env1) <- evalExpr e1 env
  (v2, env2) <- evalExpr e2 env1
  return (v1 * v2, env2)
```

```
evalExpr (Binary Pow e1 e2) env = do
  (v1, env1) <- evalExpr e1 env
  (v2, env2) <- evalExpr e2 env1
  return (v1 ** v2, env2)
```

Interactive REPL (Read-Eval-Print Loop)

Using the monadic I/O facilities developed earlier, the interactive calculator loop maintains the environment across user inputs:

```
calcLoop :: Env -> IO ()
calcLoop env = do
  putStr "calc> "
  input <- getLine
  if input `elem` ["quit", "exit"]
  then putStrLn "Exiting calculator. Goodbye."
  else case parseLine input of
    Left err -> do
      putStrLn ("Parse error: " ++ err)
      calcLoop env
    Right ast -> case evalExpr ast env of
      Left evalErr -> do
        putStrLn ("Runtime error: " ++ evalErr)
        calcLoop env
      Right (result, nextEnv) -> do
        putStrLn ("## " ++ show result)
        calcLoop nextEnv
```

This architecture neatly decouples scanning, parsing, semantic evaluation, and stateful I/O into clean, mathematically testable components.

Part III: Back-End Construction & Code Generation

Stack Machine Architecture: The 17-Instruction Virtual Machine

In many real-world language runtimes—including the Java Virtual Machine (JVM), the Python bytecode interpreter, and the WebAssembly abstract machine—programs are executed by an abstract **stack machine** rather than a register machine.

Architectural Layout: Code, Data, and Stack Memories

The virtual machine specified in the course (`mach.py`) consists of three disjoint memory areas and two control pointers:

1. `program[]`: Linear array storing executable bytecode instructions and immediate numeric operands.
2. `data[]`: Random-access array storing global variables and structured arrays indexed by integer addresses.
3. `stack[]`: LIFO (Last-In-First-Out) operand stack for arithmetic, logical evaluations, and temporary results.
4. `pc` (Program Counter): Integer register pointing to the index of the next instruction in `program[]` to be executed.
5. `sp` (Stack Pointer): Points to the next free cell in `stack[]`.

Table 0.1: Formal Specification of the 17 Minimal Machine Instructions

Category	Mnemonic	Formal Operational Semantics
Data Movement	PUSH x	pc++; stack[sp++] = program[pc++]
	LOAD	pc++; stack[sp-1] = data[stack[sp-1]]
	STORE	pc++; data[stack[sp-2]] = stack[sp-1]; sp--
	SWAP	pc++; tmp = stack[sp-1]; stack[sp-1] = stack[sp-2]; stack[sp-2] = tmp
Arithmetic & Logic	ADD	pc++; stack[sp-2] = stack[sp-2] + stack[sp-1]; sp--
	SUB	pc++; stack[sp-2] = stack[sp-2] - stack[sp-1]; sp--
	MUL	pc++; stack[sp-2] = stack[sp-2] * stack[sp-1]; sp--
	DIV	pc++; stack[sp-2] = stack[sp-2] / stack[sp-1]; sp--
	AND	pc++; stack[sp-2] = stack[sp-2] & stack[sp-1]; sp-- (bit-wise)
	OR	pc++; stack[sp-2] = stack[sp-2] stack[sp-1]; sp-- (bit-wise)
	NOT	pc++; stack[sp-1] = ~stack[sp-1]; (bit-wise complement)
Control Flow	BEZ x	pc++; pc = (stack[--sp] == 0) ? program[pc] : pc + 1
	BGZ x	pc++; pc = (stack[--sp] > 0) ? program[pc] : pc + 1
	GOTO	pc = stack[--sp]
	STOP	pc++; exit(0)
Interactive I/O	IN	pc++; stack[sp++] = read_integer()
	OUT	pc++; print(stack[--sp])

The Symbolic Assembler (**asm**)

Hand-writing numeric byte offsets for **pc** jumps is notoriously brittle. The companion assembler **asm** provides symbolic assembly with label resolution and data reservation:

1. Symbolic Jump Targets (**EQU ***):

```
loop_start EQU *
```

Defines the symbol **loop_start** as the current program address.

2. Data Allocation (**DS <size>**):

```
x DS 1      ;/ Allocates 1 integer word for variable x
arr DS 100   ;/ Allocates contiguous array of 100 words
```

3. Direct Memory Jumps:

```
PUSH loop_start
GOTO
```

4. Conditional Branching:

```
PUSH x
LOAD
BEZ exit_label
```

If the value of `x` is zero, execution jumps directly to `exit_label`. Otherwise, control continues to the subsequent instruction.

Code Generation: Compiling High-Level Languages to Stack Bytecode

With the front-end (scanning, parsing, AST construction) and target virtual machine established, we examine the backend: **code generation**. We detail how high-level structured constructs (arithmetic expressions, variable assignments, loops, conditionals, and array addressing) are translated systematically into sequence of stack machine operations.

Compiling Expressions: Infix to Postfix Stack Transformation

An arithmetic expression represented as an AST is compiled into stack code by performing a **post-order traversal** (compiling left subtree, right subtree, then emitting the binary opcode):

Theorem 4 (Stack Expression Invariant). *Let e be a well-typed scalar expression yielding value v . The compiled code sequence $\mathcal{C}(e)$ satisfies the property:*

$$\langle \text{Code}, \text{Data}, \text{Stack} \rangle \xrightarrow{\mathcal{C}(e)^*} \langle \text{Code}', \text{Data}, \text{Stack} : v \rangle \quad (0.19)$$

Executing $\mathcal{C}(e)$ pushes exactly one value (v) onto the operand stack without disturbing existing stack items or altering data memory.

Example. (Compilation of $\$2 + 3 \text{ times } 4\$$). The AST `Add (Lit 2) (Mul (Lit 3) (Lit 4))` is compiled into:

```
PUSH 2
PUSH 3
PUSH 4
MUL
ADD
```

Stepwise execution trace on operand stack:

1. PUSH 2: stack = [2]
2. PUSH 3: stack = [2, 3]
3. PUSH 4: stack = [2, 3, 4]
4. MUL: Pops 4 and 3, pushes $3 \times 4 = 12$. stack = [2, 12]
5. ADD: Pops 12 and 2, pushes $2 + 12 = 14$. stack = [14]

L-Values vs. R-Values in Variable Compilation

To compile variables, the compiler distinguishes between two semantic contexts:

- **R-Value (Right-hand value):** The value stored inside the variable. Compiled by pushing the variable address followed by `LOAD`:

$$\begin{aligned} \mathcal{C}_{R(x)} &= \text{PUSH } x \\ &\quad \text{LOAD} \end{aligned} \tag{0.20}$$

- **L-Value (Left-hand address):** The memory location where a value will be written. Compiled by pushing only the variable address:

$$\mathcal{C}_{L(x)} = \text{PUSH } x \tag{0.21}$$

An assignment statement $x := e$ is compiled as:

```

;/ Push destination address (L-value)
PUSH x
;/ Evaluate right-hand expression (R-value)
<code for e>
;/ Store value at address
STORE

```

Under STORE, the machine writes the value at `stack[sp-1]` into `data[stack[sp-2]]` and pops both items (`sp -= 2`).

Control Flow Compilation: If-Then-Else and While Loops

Because the stack machine provides only conditional branches (BEZ, BGZ) and unconditional jumps (GOTO), high-level structured control statements are lowered using unique compiler-generated label pairs.

2.3.1 Compiling if (cond) then S1 else S2

```

<code for cond>
BEZ else_lbl
<code for S1>
PUSH end_lbl
GOTO
else_lbl EQU *
<code for S2>
end_lbl EQU *

```

2.3.2 Compiling while (cond) do S

```

loop_start EQU *
<code for cond>
BEZ loop_end
<code for S>
PUSH loop_start
GOTO
loop_end EQU *

```

Array Indexing and Base-Offset Address Calculation

For an array A declared with size N ($A \text{ DS } N$), the memory address of element $A[i]$ is computed dynamically at run time:

$$\text{Address of } A[i] = \text{base}(A) + i \tag{0.22}$$

To evaluate an assignment $A[i] := e$:

```

;/ Compute destination L-value: address of A[i]
PUSH A
<code for i>
ADD
;/ Evaluate R-value expression
<code for e>
STORE

```

This elegant uniformity allows nested array accesses, multi-dimensional buffer lookups, and pointer dereferencing using identical stack primitives.

Functions, Activation Records & Runtime Memory Management

A complete programming language requires supporting procedural and functional abstraction: procedures, parameter passing, return values, local variables, and recursive invocation.

Function Calls and the Activation Record (Call Frame)

When a function f is invoked, a dedicated memory structure called an **Activation Record** (or **Stack Frame**) is allocated to track its execution state.

An activation record typically contains:

1. **Actual Parameters:** Arguments passed from the caller.
2. **Return Address:** The program location (pc) to resume execution after returning.
3. **Dynamic Link:** Pointer to the caller's frame (enabling stack unwinding).
4. **Local Variables:** Memory allocated for variables local to the active function invocation.
5. **Temporary Workspace:** Intermediate stack values during subexpression evaluation.

Parameter Passing Semantics

1. **Pass-by-Value (Call-by-Value):** The caller evaluates each argument expression to a value and copies that value into the function's local parameter slots. Mutations within the function do not alter caller memory.
2. **Pass-by-Reference:** The caller passes the memory **address** (L-value) of the variable. Any assignment inside the function directly modifies the caller's variable.
3. **Pass-by-Need (Call-by-Need / Laziness):** As implemented in Haskell, the caller passes an unevaluated thunk pointer. The function forces the value only when scrutinized, caching the evaluated result.

Compiling Recursive Functions on the Stack Machine

Consider compiling the canonical recursive factorial function:

```

function fact(n: integer): integer;
begin
  if n <= 1 then
    fact := 1
  else
    fact := n * fact(n - 1)
  end;
end;

```

To compile recursion onto our 17-instruction stack machine, function calls must preserve the return address and local state across recursive invocations.

Calling Convention:

1. **Caller Action:** Pushes the return address label onto the stack, pushes function arguments, and jumps to the function entry point:

```
PUSH ret_label
PUSH 5          ;/ Argument n
PUSH fact_entry
GOTO
ret_label EQU *
```

2. **Callee Action (Prolog):** Retrieves arguments into local frame slots or operates directly on the stack.
3. **Callee Action (Epilog):** Places the return value on the stack, fetches the saved return address, and jumps back:

```
;/ Return value is at stack top; swap with return address
SWAP
GOTO
```

Tail Call Optimization (TCO)

Definition 10 (Tail Call Optimization). A function call is in *tail position* if it is the absolute final action performed before the function returns. Tail Call Optimization replaces the standard call sequence (allocating a new stack frame) with an in-place jump (*GOTO*), reusing the existing frame and converting recursion into an $\mathcal{O}(1)$ space iterative loop.

Example. (Accumulator-Passing Tail Recursion). Contrast naive factorial with tail-recursive factorial:

```
-- Naive recursion (O(n) stack frames):
factNaive 0 = 1
factNaive n = n * factNaive (n - 1)

-- Tail-recursive accumulator (O(1) stack space with TCO):
factTail :: Integer -> Integer
factTail n = go n 1
  where
    go 0 !acc = acc
    go k !acc = go (k - 1) (k * acc)
```

In GHC and production compilers, tail calls are compiled directly into assembly jump instructions (*jmp*), matching the performance of hand-optimized imperative loops.

Advanced Topics: Type Checking, Optimizations & Native Code Generation

Beyond basic stack bytecode generation, modern compilers perform advanced static analysis, structural type checking, and optimization passes to generate high-performance native machine code.

Static Typing: The Hindley-Milner Type System & Algorithm W

Haskell and ML infer principal types without requiring explicit type annotations through the **Damas-Hindley-Milner (HM)** type inference system.

Theorem 5 (Principal Type Theorem). For every typable term e in the core ML/λ -calculus, there exists a unique *principal type scheme* $\sigma = \forall \alpha_1 \dots \alpha_n. \tau$ such that every valid type for e is an instance of σ .

Algorithm W executes type inference via three operations:

1. **Type Variable Generation:** Fresh variables $\alpha, \beta, \dots$ are assigned to unbound lambda abstractions and intermediate expressions.
2. **Constraint Generation:** Equations between types are accumulated from AST application and pattern-matching nodes.
3. **Robinson First-Order Unification:** Resolves systems of type equations $\tau_1 \cong \tau_2$ by computing the Most General Unifier (mgu) or reporting a compile-time type mismatch error.

Compiler Optimization Passes

Before emitting machine instructions, an optimizing compiler traverses intermediate code through several optimization passes:

1. **Constant Folding & Propagation:** Replaces static arithmetic subexpressions with their evaluated literals at compile time:

$$(3 \times 4) + x \xrightarrow{\text{fold}} 12 + x \quad (0.23)$$

2. **Common Subexpression Elimination (CSE):** Detects redundant evaluations of identical expressions within a basic block and replaces subsequent occurrences with references to the first result.
3. **Short-Circuit Boolean Evaluation:** Lowers logical operators **and** and **or** into conditional branches: In **A and B**, if **A** evaluates to false (0), **B** is skipped entirely.
4. **Dead Code Elimination (DCE):** Prunes unreachable instructions, unreferenced basic blocks, and assignments to dead variables that never influence observable outputs.

Lowering to Native Target Code: The x86-64 NASM Backend

The final stage of the course compiler pipeline translates abstract stack machine programs into real 64-bit Intel/AMD assembly via `to_nasm.py`.

In x86-64:

- The virtual stack pointer `sp` is mapped to the CPU hardware stack pointer register `rsp`.
- General-purpose 64-bit registers `rax`, `rbx`, `rcx`, `rdx` execute arithmetic instructions.
- System calls (`sys_write`, `sys_exit`) interact directly with the operating system kernel.

Example. (Translating Virtual Stack Instructions to x86-64 NASM Assembly). Consider how virtual machine instructions map to concrete x86-64 NASM mnemonics:

```
; Virtual: PUSH 42
push qword 42

; Virtual: ADD
pop  rbx      ; Pop right operand into rbx
pop  rax      ; Pop left operand into rax
add  rax, rbx  ; rax = rax + rbx
push rax      ; Push computed sum back to stack

; Virtual: LOAD
pop  rax      ; Pop memory address into rax
mov  rbx, [data + rax*8] ; Read 64-bit word from data segment
push rbx

; Virtual: STORE
pop  rbx      ; Pop value to store
pop  rax      ; Pop target memory address
mov  [data + rax*8], rbx ; Store into memory array

; Virtual: STOP
mov  rax, 60   ; sys_exit syscall number on Linux x86-64
```

```
xor rdi, rdi    ; Exit status 0  
syscall
```

This mapping bridges high-level functional theory with physical silicon execution, completing the end-to-end journey from λ -calculus to register hardware.

Course Laboratories, Milestones & Practical Exercises

Practical Problem Sets, Laboratory Exercises & Solutions

This appendix collects the complete laboratory exercises, milestones, and development problem sets from the course curriculum (including the parenthesized expressions parser, the scientific calculator, and the 42-stage Pascal-to-bytecode compiler), accompanied by complete formal specifications and solutions.

Problem Set 1: Lexing and Parsing Well-Parenthesized Expressions

Context: A language of well-parenthesized expressions over characters (and). Examples of valid strings: "", "()", "()()()", "(()())".

A.1.1.1 Exercise 1.1: Maximum Parenthesis Depth

Objective: Write a Happy grammar that directly computes the maximum nesting depth as an integer.

Solution:

```
Expression : {- empty -}           { 0 }
            | '(' Expression ')' Expression { max (1 + $2) $4 }
```

Verification:

- For "": matches empty rule $\rightarrow 0$.
- For "()": $1 + 0 = 1$, followed by $0 \rightarrow \max(1, 0) = 1$.
- For "()()": inner is 1, outer is $1 + 1 = 2$. Correct.

A.1.1.2 Exercise 1.2: Generalized Parse Tree Generation

Objective: Modify the grammar to produce an explicit multi-way tree (generalized rose tree) representing the nested syntactic hierarchy.

Solution:

```
data Parens = Node [Parens] deriving (Eq, Show)
```

Grammar rules:

```
Expr : {- empty -}           { [] }
      | '(' Expr ')' Expr     { Node $2 : $4 }
```

Problem Set 2: The Extended Scientific Calculator (24 Milestones)

The laboratory assignment tasks students with implementing an extensible interactive calculator in Haskell using Alex and Happy across 24 incremental milestones:

1. **Double precision floats:** Support numeric inputs such as 31.4156E-1.
2. **Binary arithmetic:** Left-associative +, -, *, /.
3. **Unary negation:** High-precedence prefix -.
4. **Transcendental functions:** sin, cos, tan, sqrt, exp, log.
5. **Power operator:** Right-associative exponentiation ^.

6. **Mathematical constants:** Pre-bound values `pi` ($\pi \approx 3.141592653589793$) and `e` ($e \approx 2.718281828459045$).
7. **Stateful variables & assignments:** `a = 2`, `b = a + 1`.
8. **Boolean logic & comparison:** Relational operators `<`, `<=`, `>`, `>=`, `==`, `!=` and logical `and`, `or`, `not` represented numerically (1.0 for true, 0.0 for false).
9. **Error resilience:** Graceful handling of division by zero and unbound variables without crashing the REPL loop.
10. **Multi-statement lines & comments:** Separating expressions with `;` and stripping comments `//` and `/* ... */`.
11. **Implicit multiplication:** Parsing algebraic syntax like `2*pi*r` as well as implicit `2 pi r`.

Problem Set 3: Pascal-to-Stack-Bytecode Compiler (42 Milestones)

The semester compilation project requires engineering an end-to-end compiler translating a Pascal-like imperative programming language into 17-instruction stack machine assembly. Below is the structured milestone roadmap with compilation strategies:

A.1.3.1 Stage I: Scalar Expressions and I/O (Milestones 1–7)

- **M1–M3: Integer literals and Display:** `print 42;` $\rightarrow$ `PUSH 42 \n OUT`.
- **M4–M6: Arithmetic Precedence:** Lowering `+`, `-`, `*`, `/` using standard post-order traversal with operator precedence (`%left '+' '-'`, `%left '*' '/'`).
- **M7: Comments:** Alex regular expressions discarding line comments `;/*` and block comments `\{[^\}]*\}`.

A.1.3.2 Stage II: Variables and Sequential Statements (Milestones 8–11)

- **M8: Variable Declaration:** `var x : integer;` emits `x DS 1` into the data segment.
- **M9–M10: Assignment and Variable References:** `x := e;` emits `PUSH x`, evaluates `e`, then emits `STORE`. Referencing `x` in an expression emits `PUSH x \n LOAD`.
- **M11: Interactive Input:** `read(x);` emits `PUSH x \n IN \n STORE`.

A.1.3.3 Stage III: Structured Control Flow (Milestones 12–15)

- **M12: Conditional Branching (if-then-else):** Generate unique labels using a state monad counter: Evaluate condition; emit `BEZ else_lbl`; compile `then` branch; emit `PUSH end_lbl \n GOTO`; emit `else_lbl EQU *`; compile `else` branch; emit `end_lbl EQU *`.
- **M14: While Loops (Euclidean GCD Algorithm):**

```
while y <> 0 do
begin
  r := x mod y;
  x := y;
  y := r;
end;
```

Lowering emits label `loop_start EQU *`, conditional test with `BEZ loop_end`, loop body, and unconditional backward jump `PUSH loop_start \n GOTO`.

- **M15: Fixed-Size Arrays (Sieve of Eratosthenes):** `var primes : array[100] of integer;` emits `primes DS 100`. Array write `primes[i] := 1;` compiles address computation `PUSH primes \n <eval i> \n ADD`, followed by `<eval 1> \n STORE`.

A.1.3.4 Stage IV: Functions and Procedures (Milestones 22–28)

- **M22–M24: Parameter Passing by Value:** Push arguments in left-to-right order; callee accesses arguments at negative stack offsets relative to frame pointer.
- **M26–M28: Recursive Activation Records (Factorial & Fibonacci):** Callee saves return address and frame pointer, allocates local variables, and restores caller state on exit via `SWAP \n GOTO`.

A.1.3.5 Stage V: Advanced Optimizations (Milestones 29–42)

- **M38: Short-circuit boolean evaluation:** Transform `A and B` into conditional jumps where `B` is not evaluated if `A` is false.
- **M41: Partial evaluation / Constant folding:** AST transformation collapsing constant subterms $\text{Lit } a + \text{Lit } b \rightarrow \text{Lit } (a + b)$.
- **M42: Array bounds checking:** Inject assertion bytecode before array accesses verifying $0 \leq \text{index} < \text{bound}$, aborting with `STOP` on violation.

Bibliography

- Aho, Alfred V., Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. 2006. *Compilers: Principles, Techniques, And Tools (2nd Edition)*. Addison-Wesley.
- Jones, Simon Peyton. 2003. *Haskell 98 Language and Libraries: The Revised Report*. Cambridge University Press.
- Mainland, Geoffrey, Roman Leshchinskiy, and Simon Peyton Jones. 2013. “Exploiting Vector Instructions with Generalized Stream Fusion.” In “ACM SIGPLAN International Conference on Functional Programming (Icfp).” Special issue, *ACM SIGPLAN International Conference on Functional Programming (ICFP)*, 37–48.
- Marlow, Simon, and Simon Peyton Jones. 2012. “The Glasgow Haskell Compiler.” *The Architecture of Open Source Applications 2*.
- Milner, Robin. 1978. “A Theory of Type Polymorphism in Programming.” *Journal of Computer and System Sciences* 17 (3): 348–75.
- Wadler, Philip. 1995. “Monads for Functional Programming.” *Advanced Functional Programming*, 24–52.